The Journal of Open Source Software

hexmb: a structured O-grid hexahedral mesh engine for morphing cardiovascular geometries

Jie Wang [1]

1 Department of Mechanical and Aerospace Engineering, University of Manchester, United Kingdom

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

hexmb builds all-hexahedral **O-grid (butterfly)** meshes and writes them directly as an OpenFOAM `constant/polyMesh` (Weller et al., 1998). An O-grid fills each cross-section with a square "H" core block surrounded by four boundary-fitted petal blocks: all cells are hexahedra, the boundary is honoured, and there is no point where the mesh converges to a singular axis. Because of that, a single O-grid mesh can be morphed through large deformation — such as a left ventricle contracting by more than half its volume every heartbeat — as one iso-topological mesh, with no remeshing between phases.

The engine is one operation applied to different inputs: sweeping a butterfly cross-section along a stack of rings builds a straight pipe, a bend, a tapered vessel, or a heart chamber. A *telescoping* variant joins coarse apex and valve caps to a fine body through non-conformal interfaces, so the body can carry the cell budget while the closing regions stay coarse enough to survive the contraction. hexmb ships with a reader for public Cardiac Atlas Project left-ventricle finite-element models (Fonseca et al., 2011; Radau et al., 2009), so the whole path from a public, CC0-licensed heart geometry to a checked hexahedral mesh is reproducible with no private data.

Statement of need

Moving-boundary computational fluid dynamics (CFD) of the cardiac left ventricle (LV) needs a volume mesh that stays valid as the cavity deforms through the beat. The natural structured mesh — rings converging to the apex, like lines of longitude on a globe — has a polar-axis singularity: cells at the apex have almost no volume and invert once the ventricle contracts. Cut-cell meshers avoid the singularity but need per-phase remeshing and give up the near-wall hexahedral layers that gradient-based quantities (for example wall shear stress) depend on. O-grid / butterfly topologies are standard in structured grid generation (Thompson et al., 1999). What has been missing is open tooling that *constructs* such a mesh from a measured surface, *morphs* it through a cardiac cycle as one mesh, and exports it straight to OpenFOAM.

hexmb fills that gap for the growing body of image-driven, prescribed-motion LV hemodynamics (Peighambari et al., 2026). Its users are researchers who have a time-resolved endocardial surface (from cine MRI, CT, or a public atlas) and want a valid moving hexahedral mesh for an OpenFOAM solver without a commercial blocking tool. The same engine serves any single-lumen, star-shaped, genus-0 geometry (pipes, bends, arteries, single chambers), so it is also a quick source of boundary-layer O-grid meshes for verification cases.

DRAFT

The Journal of Open Source Software

State of the field

Three families of tools touch this problem, and each stops short of it. Cut-cell and hex-dominant meshers (OpenFOAM's `snappyHexMesh`, cfMesh) mesh a static surface; a deforming chamber means remeshing every phase and mapping the solution across. Block-structured tools build O-grids: OpenFOAM's `blockMesh` from a hand-written block dictionary, Gmsh through transfinite and recombined blocks (Geuzaine & Remacle, 2009), and the commercial blocking tools (ICEM CFD, Pointwise, Cubit) interactively. None of them constructs the blocking from a surface stack or carries it through a cycle. Hyperbolic extrusion, as in pyHyp (Secco et al., 2021), grows a structured volume mesh from a structured surface mesh for external aerodynamics and writes CGNS, with no closed chamber, no cap, and no OpenFOAM path. Morphing libraries such as PyGeM (Tezzele et al., 2021) deform an existing mesh by free-form deformation, radial basis functions, or inverse distance weighting and do not build one. The cardiac and vascular toolchains — SimVascular (Updegrove et al., 2017), VMTK (Antiga et al., 2008), meshtool (Neic et al., 2020) — produce tetrahedral or hybrid meshes from images and are the right tools for branching vessels and electrophysiology; none makes an all-hexahedral O-grid or an iso-topological moving mesh.

`hexmb` is the combination: it constructs the O-grid from a stack of cross-sections, morphs it through the cycle, checks it, and writes the OpenFOAM `polyMesh`. It contributes to the OpenFOAM ecosystem in place of competing with it: the output is native, `checkMesh` is the verification step, and the telescoping assembly uses OpenFOAM's own non-conformal coupling (Farrell & Maddison, 2011). The morphing step uses the standard radial-basis-function formulation (Boer et al., 2007) and the quality gate the standard scaled Jacobian (Knupp, 2001); the contribution is the construction and the pipeline around them.

Software design

The central trade-off is structured against unstructured. A structured O-grid gives validity under deformation, graded wall layers, and a block structure that later solvers can exploit; the price is the applicability domain: one connected lumen whose cross-sections are star-shaped about their centre, swept along a gentle centreline. `hexmb` accepts that limit explicitly (branches and junctions are out of scope) and states the conditions of use in its documentation.

The second trade-off is the apex. A single O-grid over a whole ventricle inverts once the cavity has more than about fifty cells across at end-systole, because the apex cells are squeezed by the contraction. The telescoping template answers this by building three zones — a coarse apex cap, a fine body, a coarse base cap — and joining them with non-conformal interfaces, so the body reaches about a million cells while the caps stay under the pinch limit. The cost is a dependency: assembling the coupled case needs OpenFOAM (`mergeMeshes`, `createNonConformalCouples`); the per-zone build and the quality gate need only numpy.

The third choice is a small, dependency-light core. Block assembly, node deduplication by union-find, Coons transfinite interpolation, the scaled-Jacobian gate, and the `polyMesh` writer are pure numpy; SciPy is optional and speeds the deduplication through connected components, so a boundary-layer pipe of 1.2 million hexahedra builds in about two seconds on one core and four million in about nine. The writer orients every face geometrically, outward from the cell centroid, so export is correct whatever the index handedness of a block. For motion, two strategies are offered: radial-basis-function interpolation of the wall displacement for moderate motion, and *resurfacing* for large deformation, which moves the surface and refills the interior by transfinite interpolation; each phase is passed through the same validity gate.

The package is organised as `core` (blocks, multi-block mesh, topology, transfinite interpolation), `assembly` (butterfly O-grid, sweep, non-conformal assembler), `services` (quality, OpenFOAM export, smoothing, morph), `io` (Cardiac Atlas Project reader, STL surface export), and `templates` (pipe, elbow, STL tube, single-chamber LV, telescoping LV), behind a command-

DRAFT

line tool (`hexmb pipe|elbow|cap|lv|stl|verify`) and a small Python API. A pytest suite of 19 tests covers exactness of the transfinite map, two-block deduplication and export, the O-grid node-count formula, wall grading and boundary layers, pipe and elbow validity, LV patch tagging, the telescoping zones, and the assembler interface; it runs in continuous integration on every push. A known failure mode is documented: a subject-specific base with a strongly non-planar valve plane can leave inverted cells at the base cap, and the template then needs its core fraction and apex trim re-tuned for that surface.

Research impact statement

The public demonstration is reproducible from CC0 data: the shipped script takes one Cardiac Atlas Project model to an endocardial surface, an STL, and a telescoping hexahedral mesh, and the repository shows that mesh morphing through the 20 phases of the cardiac cycle as one iso-topological mesh with zero inverted cells and a clean `checkMesh`.

In the author's own research the engine underlies a cine-MRI-to-CFD pipeline for the moving left ventricle. Its production mesh, a conformal 805,200-cell LV built on the `hexmb` O-grid cross-section, block assembly, and `polyMesh` writer (with a split-hexahedron transition that is still research code), carries every run of a GPU pressure-solve port of OpenFOAM 12 and the comparison of simulated filling against each subject's own 4D-flow MRI for two subjects; the same mesh supplied the near-wall shell of an overset variant and the moving surface of a lattice-Boltzmann model. On that mesh the moving-wall solve runs through the filling phase, the interval of largest wall motion, with continuity errors near 1e-6 and no apex inversion, which is the property the O-grid was built for.

The software is young: first public release 18 September 2026, one author, no external users yet. The community-readiness signals in place are an MIT licence, continuous integration, a test suite, a contributing guide, a worked example on public data, and a stated applicability domain; the split-hexahedron transition and a conforming junction mesher are the next items on the public roadmap.

AI usage disclosure

A generative AI coding assistant (Claude Code, Anthropic) was used throughout the project: to draft code and tests from the author's specifications, to write documentation, and to draft this paper. The author designed the method and the architecture (the O-grid topology and sweep, the telescoping zones, the morph strategies, and the quality gate), directed and reviewed every change, and is responsible for the result. Correctness was checked by means independent of the assistant: the test suite runs in continuous integration; every exported mesh passes the scaled-Jacobian gate and OpenFOAM's `checkMesh`; the full-cycle morph of the public left ventricle and the moving-wall flow runs on the derived meshes exercised the code end to end.

Acknowledgements

The left-ventricle demonstration uses the Cardiac Atlas Project's Sunnybrook Cardiac Data, released into the public domain (CC0) (Fonseca et al., 2011; Radau et al., 2009). The author received no specific funding for this work.

References

Antiga, L., Piccinelli, M., Botti, L., Ene-Iordache, B., Remuzzi, A., & Steinman, D. A. (2008). An image-based modeling framework for patient-specific computational hemodynamics.

DRAFT

Medical & Biological Engineering & Computing, *46*(11), 1097–1112. https://doi.org/10.1007/s11517-008-0420-1

Boer, A. de, Schoot, M. S. van der, & Bijl, H. (2007). Mesh deformation based on radial basis function interpolation. *Computers & Structures*, *85*(11–14), 784–795. https://doi.org/10.1016/j.compstruc.2007.01.013

Farrell, P. E., & Maddison, J. R. (2011). Conservative interpolation between volume meshes by local Galerkin projection. *Computer Methods in Applied Mechanics and Engineering*, *200*(1–4), 89–100. https://doi.org/10.1016/j.cma.2010.07.015

Fonseca, C. G., Backhaus, M., Bluemke, D. A., Britten, R. D., Chung, J. D., Cowan, B. R., Dinov, I. D., Finn, J. P., Hunter, P. J., Kadish, A. H., Lee, D. C., Lima, J. A. C., Medrano-Gracia, P., Shivkumar, K., Suinesiaputra, A., Tao, W., & Young, A. A. (2011). The Cardiac Atlas Project—an imaging database for computational modeling and statistical atlases of the heart. *Bioinformatics*, *27*(16), 2288–2295. https://doi.org/10.1093/bioinformatics/btr360

Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A 3-D finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, *79*(11), 1309–1331. https://doi.org/10.1002/nme.2579

Knupp, P. M. (2001). Algebraic mesh quality metrics. *SIAM Journal on Scientific Computing*, *23*(1), 193–218. https://doi.org/10.1137/S1064827500371499

Neic, A., Gsell, M. A. F., Karabelas, E., Prassl, A. J., & Plank, G. (2020). Automating image-based mesh generation and manipulation tasks in cardiac modeling workflow using Meshtool. *SoftwareX*, *11*, 100454. https://doi.org/10.1016/j.softx.2020.100454

Peighambari, S. B., Mukherjee, T., Mendiola, E. A., Darwish, A., Criscione, J. C., Timmins, L. H., Pettigrew, R. I., Shah, D. J., & Avazmohammadi, R. (2026). Computational modeling of left ventricular flow using PC-CMR-derived four-dimensional wall motion. *International Journal of Engineering Science*, *224*, 104499. https://doi.org/10.1016/j.ijengsci.2026.104499

Radau, P., Lu, Y., Connelly, K., Paul, G., Dick, A. J., & Wright, G. A. (2009). Evaluation framework for algorithms segmenting short axis cardiac MRI. *The MIDAS Journal — Cardiac MR Left Ventricle Segmentation Challenge*. https://doi.org/10.54294/g80ruo

Secco, N. R., Kenway, G. K. W., He, P., Mader, C., & Martins, J. R. R. A. (2021). Efficient mesh generation and deformation for aerodynamic shape optimization. *AIAA Journal*, *59*(4), 1151–1168. https://doi.org/10.2514/1.J059491

Tezzele, M., Demo, N., Mola, A., & Rozza, G. (2021). PyGeM: Python geometrical morphing. *Software Impacts*, *7*, 100047. https://doi.org/10.1016/j.simpa.2020.100047

Thompson, J. F., Soni, B. K., & Weatherill, N. P. (Eds.). (1999). *Handbook of grid generation*. CRC Press.

Updegrove, A., Wilson, N. M., Merkow, J., Lan, H., Marsden, A. L., & Shadden, S. C. (2017). SimVascular: An open source pipeline for cardiovascular simulation. *Annals of Biomedical Engineering*, *45*(3), 525–541. https://doi.org/10.1007/s10439-016-1762-8

Weller, H. G., Tabor, G., Jasak, H., & Fureby, C. (1998). A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, *12*(6), 620–631. https://doi.org/10.1063/1.168744

DRAFT